

TD de Programmation Orientée Objet

Licence d'Informatique

Semaine du 13 avril 2026

Design pattern Builder

Le design pattern Builder est utile pour créer des instances d'une classe possédant de nombreux paramètres, ou bien dont l'initialisation peut être complexe. L'application du design pattern Builder permet par ailleurs de cacher le nom de la classe effectivement instanciée et par conséquent de limiter la dépendance à une implémentation particulière.

Prenons par exemple une classe représentant une UE (Unité d'enseignement) de licence :

```
public class UniteEnseignement {
    private String name;
    private String description;
    private int ects = 4;
    private int coefP1 = 1;
    private int coefP2 = 2;
    private int coefTP = 1;
    private int numSemestre = 1;

    public UniteEnseignement(String name, String description,
                             int ects, int coefP1, int coefP2,
                             int coefTP, int numSemestre) {

        // Init attributs
    }
}
```

L'appel d'un constructeur ressemblerait à :

```
UniteEnseignement poo2 = new UniteEnseignement("POO2",
                                                "Design_Patterns",
                                                4, 1, 2, 1, 4);
```

Avec un risque important de confusion entre les différents paramètres numériques.

L'application du design pattern Builder consiste à créer une classe dont le rôle unique sera de créer une instance de la classe :

- la classe builder, possède un constructeur dont les paramètres sont ceux qui n'ont pas de valeur par défaut
- pour chaque paramètre optionnel (i.e. avec une valeur par défaut) on définit une méthode pour enregistrer sa valeur
- on définit une méthode build() qui crée effectivement l'instance et la renvoie
- dans la classe UniteEnseignement on remplace le/les constructeur(s) par un unique constructeur ayant en paramètre le builder.

```
public class UEBuilder {
    String name;
    String description;
    int ects = 4;
    int coefP1 = 1;
    int coefP2 = 2;
    int coefTP = 1;
    int numSemestre = 1;

    public UEBuilder(String name, String description) {
```

```

        if (name == null || description == null) {
            throw new NullPointerException();
        }
        this.name = name;
        this.description = description;
    }

    public UEBuilder withEcts(int val) {
        ects = val;
        return this;
    }

    public UEBuilder withCoefP1(int val) {
        coefP1 = val;
        return this;
    }

    // idem avec les autres parametres

    public UniteEnseignement build() {
        return new UniteEnseignement(this);
    }
}

```

En complément il faut donc définir dans la classe `UniteEnseignement` un constructeur prenant en paramètre une instance de `UEBuilder`. Ce constructeur ne sera utilisé que par la méthode `build()` de la classe `UEBuilder` et on va donc lui affecter une visibilité restreinte package private :

```

public class UniteEnseignement {
    private String name;
    private String description;
    private int ects = 4;
    private int coefP1 = 1;
    private int coefP2 = 2;
    private int coefTP = 1;
    private int numSemestre = 1;

    // Constructeur avec visiblite package private
    // aucun autre constructeur n'est necessaire
    // la creation d'une instance devra donc imperativement
    // utiliser le builder
    //
    UniteEnseignement(UEBuilder builder) {
        name = builder.name;
        description = builder.description;
        ects = builder.ects;
        coefP1 = builder.coefP1;
        coefP2 = builder.coefP2;
        coefTP = builder.coefTP;
        numSemestre = builder.numSemestre;
    }
}

```

La création d'une instance devient :

```

UniteEnseignement poo2 = new UEBuilder("POO2", "Design_Patterns")
    .withNumSemestre(4)
    .build();

```

Exercice 1 Définition d'une classe `ListViewBuilder` pour l'interface `ListView`.

L'interface `ListView<E>` vue la semaine dernière permet de créer un objet capable d'afficher une liste numérotée d'objets de type `E`. Pour obtenir une instance d'une classe implémentant l'interface `ListView<E>` on a utilisé la méthode :

```
public static <T> ListView<T> buildMenu(Iterable<T> items)
```

Nous allons construire une classe `ListeViewBuilder` permettant de remplacer cette méthode static.

Une des classes implémentant l'interface `ListView` est la classe `MenuView` qui possède les attributs suivants :

```
class MenuView<E> implements ListView<E> {  
    private String title; // le titre  
                        // Valeur par défaut " "  
    private String header; // texte a afficher sous le titre  
                        // Valeur par défaut " "  
    private List<E> list; // Les objets a afficher pour le choix  
                        // parametre obligatoire  
    private String footer; // texte a afficher sous la liste  
                        // Valeur par défaut " "
```

1. Définissez la classe `ListeViewBuilder<E>` de manière à ce qu'elle crée une instance de `MenuView<E>`. La méthode `build()` de la classe `ListeViewBuilder<E>` devra avoir l'entête :

```
public ListView<E> build()
```

2. Définissez dans la classe `MenuView` un constructeur prenant en paramètre une instance de `ListeViewBuilder`.
3. Donnez le code permettant de créer une `ListView<E>` à l'aide de la classe `ListeViewBuilder<E>`.

Exercice 2 Prise en compte d'une seconde classe implémentant `ListView`.

`GUIView<E>` est une autre classe implémentant l'interface `ListView` qui affiche une liste de choix par l'intermédiaire d'une interface graphique, pour cette seconde classe nous allons donc avoir besoin d'un builder spécifique pour prendre en compte les paramètres spécifiques à cette autre classe et créer une instance de cette classe au lieu d'un `MenuView<E>`. En particulier, la classe `GUIView<E>` possède un paramètre de type `JFrame` qui représente la fenêtre dans laquelle doit s'afficher la liste.

Pour prendre en compte les paramètres communs aux deux classes `MenuView<E>` et `GUIView<E>`, nous allons créer une classe abstraite `AbstractListView` et une classe abstraite `AbstractListViewBuilder`. Les deux classes `MenuView<E>` et `GUIView<E>` seront alors définies par héritage de cette classe abstraite et nous allons définir une nouvelle classe builder pour chacune d'entre elles héritant de `AbstractListViewBuilder` : les classes `MenuViewBuilder<E>` et `GUIViewBuilder<E>`.

1. Définissez les classes `AbstractListView` et `AbstractListViewBuilder`
2. Définissez les classes `MenuViewBuilder<E>` et `GUIViewBuilder<E>`